

# Práctica 2C – Vulnerabilidades de Autenticación

Seguridad en el Desarrollo de Aplicaciones

Alumno: Luz Celeste López Rivera

Docente: Gerardo Ramírez Villarreal

## Objetivo

El objetivo de esta práctica fue implementar un sistema de autenticación vulnerable utilizando ASP.NET Core MVC con el fin de analizar fallos de seguridad relacionados con autenticación insegura y SQL Injection.

## Desarrollo de la práctica

### 1. Configuración de sesiones

Primero se habilitó el uso de sesiones dentro de Program.cs utilizando `AddSession()` y `UseSession()`.

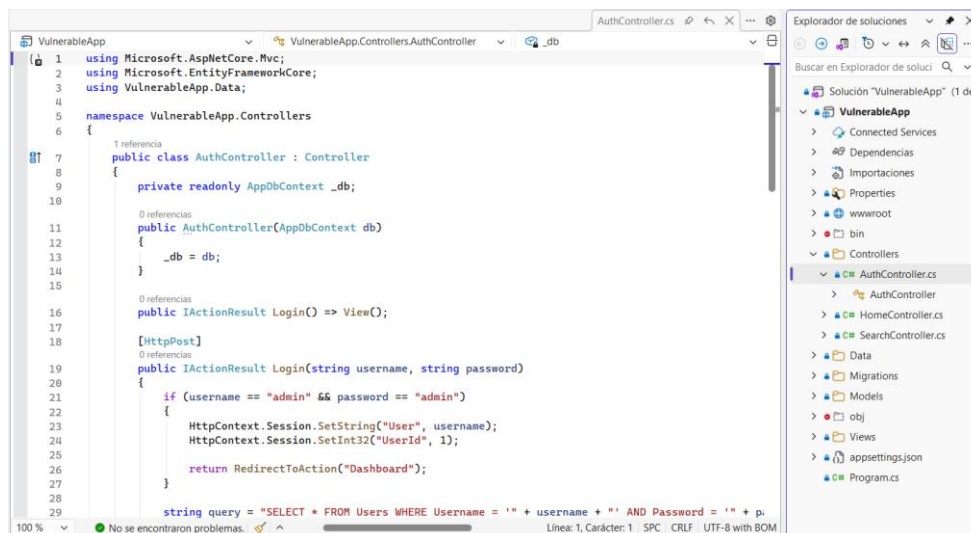
```
10 builder.Services.AddAuthentication();
11 builder.Services.AddSession();
12
app.UseSession();
```

Esto permitió almacenar información básica del usuario autenticado dentro de la aplicación.

### 2. Creación de AuthController

Se creó un controlador llamado `AuthController` encargado de:

- Mostrar formulario de login
- Validar credenciales
- Crear sesión
- Mostrar dashboard
- Cerrar sesión



### 3. Implementación de autenticación vulnerable

Dentro del método Login se implementaron credenciales hardcodeadas:

```
if (username == "admin" && password == "admin")
```

Posteriormente se agregó una consulta SQL vulnerable:

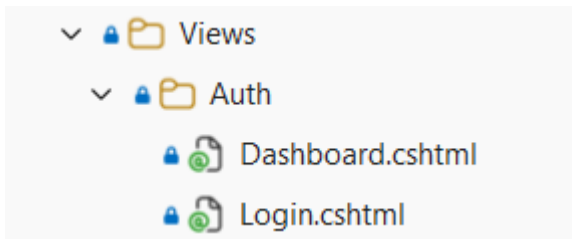
```
string query = "SELECT * FROM Users WHERE Username = '" + username + "' AND Password = '" + password + "'";
```

La consulta era ejecutada mediante:

```
var user = _db.Users.FromSqlRaw(query).FirstOrDefault();
```

### 4. Creación de vistas

Se crearon las siguientes vistas:



La vista Login permite ingresar credenciales y la vista Dashboard muestra información del usuario autenticado.

## Pruebas realizadas

### Login válido

Se realizaron pruebas utilizando:

### Login Vulnerable

Usuario:

Contraseña:

### Dashboard

Bienvenido admin

- Email: admin@test.com
- Balance: \$1000.00

[Cerrar sesión](#)

La aplicación permitió el acceso correctamente.

### Login inválido

Se probaron credenciales incorrectas y el sistema mostró el mensaje:

### Login Vulnerable

Usuario:

Contraseña:

### Login Vulnerable

Usuario:

Contraseña:

Usuario/contraseña inválido

### SQL Injection en autenticación

Se realizó una prueba utilizando:

### Login Vulnerable

Usuario:

Contraseña:

### Dashboard

Bienvenido user1

- Email: user@test.com
- Balance: \$500.00

[Cerrar sesión](#)

Esto permitió alterar la consulta SQL original demostrando una vulnerabilidad de autenticación insegura.

## Explicación técnica de la vulnerabilidad

La aplicación presenta múltiples vulnerabilidades de autenticación:

- Uso de credenciales hardcodeadas
- Contraseñas almacenadas en texto plano
- Consultas SQL sin parametrización
- Ausencia de hash de contraseñas
- Uso inseguro de sesiones

La vulnerabilidad principal corresponde nuevamente a SQL Injection (CWE-89).

Al concatenar directamente los datos ingresados por el usuario dentro de la consulta SQL, es posible alterar la lógica original de autenticación.

## Resultados obtenidos

La aplicación permitió:

- Inicio de sesión exitoso
- Manejo básico de sesiones
- Acceso al dashboard
- Demostración de autenticación vulnerable
- Ejecución de SQL Injection sobre el login

## Tabla de resultados de prueba

| Prueba realizada             | Datos utilizados    | Resultado                          |
|------------------------------|---------------------|------------------------------------|
| Login válido                 | admin / admin       | Acceso exitoso al Dashboard        |
| Login válido usuario semilla | user1 / 123456      | Acceso exitoso                     |
| Login inválido               | aaa / 123           | Usuario/contraseña inválido        |
| SQL Injection                | ' OR '1'='1         | Vulnerabilidad reproducida         |
| Acceso al Dashboard          | Usuario autenticado | Información mostrada correctamente |

## Conclusión

La práctica permitió comprender cómo errores comunes en autenticación pueden comprometer completamente la seguridad de una aplicación web.

También permitió observar los riesgos de:

- Consultas SQL dinámicas
- Contraseñas sin cifrado
- Credenciales hardcodeadas
- Manejo inseguro de sesiones

Finalmente, se logró implementar un entorno vulnerable funcional para fines educativos y de análisis de seguridad.